

CS1540 Algorithmic design

Steven Chaplick, David Mestel

Homework 2. Dynamic Programming

These exercises are simply for additional practice and include an implementation task that can be automatically tested via CodeGrade.

For testing via Codegrade see the link [Canvas](#). Your program needs to read a filename from the terminal, read the input from the named file, and print the output (and nothing else) to the terminal. The Canvas page contains an example `.java` file demonstrating the I/O. Note that **the file you upload must have the same name and main class as the example, otherwise the testing will fail**. An example input file and its expected output is also included. You can submit as many times as you want, subject to a limit of one submission every 5 minutes. To ensure your implementation is fast, the runtime time limit for each test case is 20 seconds.

1 ‘Anchored’ common subsequences

Recall the definition of longest common subsequence (LCS) from the lecture: the LCS of strings s_1, s_2 is the longest string s such that s is a subsequence of both s_1 and s_2 , i.e., we can get s by starting at some point in s_i , reading through, perhaps skipping some letters, and then stopping at some point.

Suppose now our alphabet contains a special ‘anchor’ character $*$ which binds ‘more strongly’ than ordinary letters: concretely, a string s is an anchored common subsequence if no $*$ is skipped when reading through the relevant substring of each s_i (note: $*$ s can occur outside the substring of s_i where s occurs).

For example, suppose that $s_1 = A * BCD*$ and $s_2 = B * CAD$. We would *not* be allowed to take the subsequence BC (because this requires skipping a $*$ in s_2), but we *would* be allowed to take the subsequence $*CD$, which is the longest anchored common subsequence and has length 3.

2 Examples

$s_1 = A * BF * CB$
 $s_2 = F * CAB * D$
LACS = $F * CB$

$s_1 = BFA * B$
 $s_2 = CB * A * BB$
LACS = $A * B$

$s_1 = ACCBDFBA * BA$
 $s_2 = CBAAF * BAA$
LACS = $CBF * BA$ or $CBA * BA$

$s_1 = A * B * CD*$
 $s_2 = B * CAD$
LACS = $B * CD$

$s_1 = *AC*$
 $s_2 = *BD*$
LACS = $**$

3 Your Tasks

1. Provide a dynamic programming algorithm (in pseudocode) to find a longest anchored common subsequence of two strings. Justify the correctness and analyze the running time of your algorithm.
2. Implement your algorithm. The input file will have two lines, each consisting of a string over alphabet $A, B, \dots, Z, *$ of length at most 2,000 characters. Your program should output a single integer n , the length of the longest anchored common subsequence.

Name your program `Anchored.java` and submit it to Codegrade for testing.

4 Hints

Start from the approach to Longest Common Subsequence (as in the lecture).

- Try running it by hand on strings that do not contain *.
- Then, try the algorithm on strings with *.

By observing where it goes wrong, try to modify the algorithm so that it will work.

Make some new test cases for your new algorithm to see if it really works. Start with small cases where you can see the answer by inspection (just looking at the problem) and which are easy to work out by hand. Continue trying different cases until you cannot think of a case where your algorithm gives the wrong answer.